

Chapter 2

Data Sources and APIs

Types of Data

- Structured Data
- Unstructured Data
- Semi-Structured Data

Structured Data

- Organized in rows and columns
 - Fixed schema
 - Examples: CSV, Excel, database tables

CSV Files

- Structured tabular data
- Widely used in data analysis
- Handled using pandas

Pandas introduction

Pandas is a Python library used for working with data sets.

It has functions for analyzing, cleaning, exploring, and manipulating data.

Install Pandas:

```
C:\Users\Your_pc_name>pip install pandas
```

Pandas Series

A Pandas Series is like a column in a table.

It is a one-dimensional array holding data of any type.

```
import pandas as pd  
a = [10,20,30]  
myvar = pd.Series(a)  
print(myvar)
```

```
0    10  
1    20  
2    30  
dtype: int64
```

Pandas Labels

It is a index given to each rows.

If nothing else is specified, the values are labeled with their index number. First value has index 0, second value has index 1 etc.

```
import pandas as pd
a = [10, 20, 30]
myvar = pd.Series(a, index = ["a", "b", "c"])
print(myvar)
```

```
a    10
b    20
c    30
dtype: int64
```

Pandas DataFrame

A Pandas DataFrame is a 2 dimensional data structure, like a 2 dimensional array, or a table with rows and columns.

```
import pandas as pd

data = {
    "Column_1": [10, 20, 30],
    "Column_2": [40, 50, 60]
}

df = pd.DataFrame(data)
print(df)
```

	Column_1	Column_2
0	10	40
1	20	50
2	30	60

Reading csv file

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df)
print(type(df))
# print(df.to_string())
```

Reading JSON

```
import pandas as pd  
df = pd.read_json('datas.json')  
print(df.to_string())
```

Working with Dictionaries

```
data = {  
    "Duration":{  
        "0":60,  
        "1":60,  
        "2":60,  
        "3":45,  
        "4":45,  
        "5":60  
    },  
    }
```

```
    "Pulse":{  
        "0":110,  
        "1":117,  
        "2":103,  
        "3":109,  
        "4":117,  
        "5":102  
    },  
    }
```

```
    "Maxpulse":{  
        "0":130,  
        "1":145,  
        "2":135,  
        "3":175,  
        "4":148,  
        "5":127  
    },  
    }
```

```
    "Calories":{  
        "0":409,  
        "1":479,  
        "2":340,  
        "3":282,  
        "4":406,  
        "5":300  
    }  
}  
  
df = pd.DataFrame(data)  
  
print(df)
```

Similarly....

Read CSV File (Python)

```
import pandas as pd  
df = pd.read_csv('data.csv')  
print(df.head())
```

Write CSV File (Python)

```
df.to_csv('output.csv', index=False)
```

Excel Files

- Spreadsheet data format
- Supports multiple sheets
- Requires openpyxl library

Read Excel File (Python)

```
import pandas as pd  
df = pd.read_excel('data.xlsx',  
sheet_name='Sheet1')
```

Write Excel File (Python)

```
df.to_excel('output.xlsx', index=False)
```

JSON Files

- Semi-structured data format
- Common in web APIs
- Hierarchical structure

Read JSON File (Python)

```
import pandas as pd  
df = pd.read_json('data.json')  
print(df)
```

Write JSON File (Python)

```
df.to_json('output.json', orient='records',  
indent=4)
```

Text Files

- Unstructured data format
- No fixed schema
- Used for logs and documents

Read Text File (Python)

```
with open('sample.txt', 'r') as file:  
    content = file.read()  
    print(content)
```

Write Text File (Python)

```
with open('output.txt', 'w') as file:  
    file.write('This is unstructured text  
data')
```

Format Comparison

- CSV / Excel – structured data analysis
- JSON – web services and APIs
- Text – logs and natural language processing

Conclusion

- Different data formats require different handling methods
 - Python provides pandas and built-in file handling
 - Essential concept for data analysis and programming

Database Access

Relational & Non-Relational

Databases

What is Database Access?

- Database access refers to how applications connect to, read from, write to, update, and manage data stored in databases. It is done using database drivers, connectors, or ORMs.

Types of Databases

- 1. Relational Databases (SQL)
 - Structured data (tables, rows, columns)
 - Fixed schema
 - Use SQL language
 - Examples: MySQL, PostgreSQL, Oracle, SQLite
- 2. Non-Relational Databases (NoSQL)
 - Semi-structured or unstructured data
 - Flexible schema
 - Different data models
 - Examples: MongoDB, Redis, Cassandra

Relational Database Characteristics

- - Data stored in tables
 - Primary & Foreign Keys
 - ACID properties
 - Strong consistency
 - Suitable for financial and transactional systems

Non-Relational Database Characteristics

- - Document, Key-Value, Column, Graph models
 - High scalability
 - Schema-less or flexible schema
 - BASE properties
 - Suitable for big data & real-time applications

Python & Relational Databases

- Common Python libraries:
 - sqlite3 (built-in)
 - psycopg2 (PostgreSQL)
 - mysql-connector-python
 - SQLAlchemy (ORM)

Workflow:

Connect → Execute SQL → Fetch Data → Close Connection

SQLite Database Access (Python)

- ```
import sqlite3

conn = sqlite3.connect("example.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS users (
 id INTEGER PRIMARY KEY,
 name TEXT,
 age INTEGER
)
""")

cursor.execute("INSERT INTO users (name, age) VALUES (?, ?)", ("Dhiraj", 30))

conn.commit()

cursor.execute("SELECT * FROM users")
rows = cursor.fetchall()

for row in rows:
 print(row)

conn.close()
```

# Using ORM (SQLAlchemy)

- ORM maps database tables to Python classes.

Benefits:

- Less SQL code
- Database independent
- Cleaner & maintainable code

# SQLAlchemy Example

- ```
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import declarative_base, sessionmaker
```

```
engine = create_engine("sqlite:///example.db")
Base = declarative_base()
```

```
class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    name = Column(String)
    age = Column(Integer)
```

```
Base.metadata.create_all(engine)
```

```
Session = sessionmaker(bind=engine)
session = Session()
```

```
user = User(name="Ram", age=25)
session.add(user)
session.commit()
```

```
users = session.query(User).all()
for u in users:
    print(u.name, u.age)
```

Python & Non-Relational Databases

- Common Python libraries:
 - pymongo (MongoDB)
 - redis-py (Redis)

Workflow:

Connect → Select Database/Collection → CRUD Operations

MongoDB (Document Database)

- - Stores data in JSON-like documents (BSON)
 - No fixed schema
 - High performance & scalability
 - Widely used with Python web frameworks

MongoDB Access with PyMongo

- `from pymongo import MongoClient`

```
client = MongoClient("mongodb://localhost:27017/")
```

```
db = client["testdb"]  
collection = db["users"]
```

```
collection.insert_one({  
    "name": "Sita",  
    "age": 28  
})
```

```
users = collection.find()
```

```
for user in users:  
    print(user)
```

SQL vs NoSQL Comparison

- Relational (SQL):
 - Fixed schema
 - Vertical scaling
 - Strong consistency

Non-Relational (NoSQL):

- Flexible schema
- Horizontal scaling
- Eventual consistency

When to Use Which?

- Use Relational Databases when:
 - Data is structured
 - Transactions are critical

Use Non-Relational Databases when:

- Data is large & unstructured
- High scalability is required

Conclusion

- Both relational and non-relational databases are important.
Python provides powerful libraries for accessing both.
Choosing the right database depends on application requirements.

Accessing and Processing Data from APIs

REST & SOAP APIs with Python

What is an API?

- API stands for Application Programming Interface
- Allows communication between client and server
- Used to exchange data between applications
- Common in web, mobile, and data science projects

API Communication Flow

Client Application

API Request (HTTP)

Server Processing

API Response

Client Uses Data

REST API Overview

- Uses HTTP protocol (GET, POST, PUT, DELETE)
- Data format: JSON
- Lightweight and fast
- Most widely used API type

Python Code: REST API (GET Request)

- `import requests`

```
url = "https://jsonplaceholder.typicode.com/posts/1"  
response = requests.get(url)
```

```
data = response.json()  
print(data)
```

Processing REST API Response

- ```
title = data['title']
body = data['body']
```

```
print("Title:", title)
print("Body:", body)
```

# REST API with Parameters

- `params = {"userId": 1}`

```
response = requests.get(
 "https://jsonplaceholder.typicode.com/posts",
 params=params
)
```

```
posts = response.json()
print(len(posts))
```

# REST API POST Request (Python)

- ```
payload = {  
    "title": "API Learning",  
    "body": "Learning REST API with Python",  
    "userId": 1  
}  
  
response = requests.post(  
    "https://jsonplaceholder.typicode.com/posts",  
    json=payload  
)  
  
print(response.json())
```

SOAP API Overview

- SOAP stands for Simple Object Access Protocol
- Uses XML format
- Highly structured and secure
- Used in banking and enterprise systems

Python Code: SOAP API Example

- `from zeep import Client`

```
wsdl_url = "http://www.dneonline.com/calculator.asmx?WSDL"  
client = Client(wsdl=wsdl_url)
```

```
result = client.service.Add(10, 20)  
print(result)
```

SOAP API Working Flow

Client Creates XML Request

SOAP Envelope Sent

Server Processes Request

XML Response Returned

Client Parses Response

API Error Handling (Python)

- `response = requests.get(url)`

```
if response.status_code == 200:
```

```
    print("Request Successful")
```

```
else:
```

```
    print("Error:", response.status_code)
```

REST vs SOAP Comparison

- REST: Fast, simple, JSON-based
- SOAP: Secure, XML-based
- REST for web/mobile apps
- SOAP for enterprise systems

Summary

- APIs enable system communication
- REST APIs are most common
- SOAP APIs are secure and strict
- Python simplifies API access

Web Scraping with Python

Requests + BeautifulSoup

Author: Dhiraj Pyakurel

Objective

- Understand web scraping workflow
- Use requests and BeautifulSoup
- Scrape real website data
- Save scraped data into Excel

Libraries Required

- requests → Send HTTP requests
- bs4 → Parse HTML
- pandas → Store & export data
- openpyxl → Write Excel files

Target Website (Case Study)

- Website: <https://quotes.toscrape.com>
- Data extracted:
- Quote text
- Author
- Tags

Step 1: Import Libraries

- `import requests`
- `from bs4 import BeautifulSoup`
- `import pandas as pd`

Step 2: Send HTTP Request

- `url = 'https://quotes.toscrape.com/'`
- `response = requests.get(url)`
- `if response.status_code != 200:`
 `print('Request failed')`
 `exit()`

Step 3: Parse HTML

- `soup = BeautifulSoup(response.text, 'html.parser')`

Step 4: Initialize Data Variable

(IMPORTANT)

- `data = []` # This list will store scraped data

Step 5: Extract Data

```
quotes = soup.find_all('div', class_='quote')
```

```
for quote in quotes:
```

```
    text = quote.find('span', class_='text').get_text(strip=True)
```

```
    author = quote.find('small', class_='author').get_text(strip=True)
```

```
    tags = [tag.get_text(strip=True) for tag in quote.find_all('a', class_='tag')]
```

```
    data.append({
```

```
        'Quote': text,
```

```
        'Author': author,
```

```
        'Tags': ', '.join(tags)
```

```
    })
```

Step 6: Convert to DataFrame

- `df = pd.DataFrame(data)`
- `print(df.head())`

Step 7: Save Data to Excel

- `df.to_excel('quotes.xlsx', index=False)`
- `print('Excel file created successfully')`

Output File

- quotes.xlsx
- Contains all scraped quotes, authors, and tags

Ethical Web Scraping

- Avoid frequent requests
- Scrape only public data
- Follow website terms

Handling Large Datasets in Python

Chunking and Lazy Evaluation

Author: Dhiraj Pyakurel

Why Large Datasets Are a Problem

- Data may not fit into RAM
- Causes slow performance or crashes
- Common in CSV, logs, web scraping, databases

Key Concepts

- Chunking:

Process data in small pieces

- Lazy Evaluation:

Load and compute data only when needed

What is Chunking?

- Chunking means reading data in fixed-size blocks
- Instead of loading entire file:
- → Load 10,000 rows at a time

Chunking with pandas

- `pd.read_csv(..., chunksize=10000)`
- Returns an iterator, not full DataFrame

Chunking Example Code

- `import pandas as pd`
- `chunks = pd.read_csv('large.csv', chunksize=10000)`
- `for chunk in chunks:`
- `print(chunk.shape)`

What is Lazy Evaluation?

- Data is not loaded immediately
- Computation happens only when required
- Saves memory and time

Lazy Evaluation Example (Generator)

```
def read_large_file(path):  
    with open(path) as f:  
        for line in f:  
            yield line
```

Real Case Study

- Case: Scraped 1 million job listings

- Tasks:

- Count jobs per location

- Avoid memory overflow

Case Study: Chunk Processing

```
job_counts = {}
```

```
for chunk in pd.read_csv('jobs.csv', chunksize=50000):
```

```
    for city in chunk['location']:
```

```
        job_counts[city] = job_counts.get(city, 0) + 1
```

Saving Aggregated Result

```
df = pd.DataFrame(job_counts.items(), columns=['City','Jobs'])  
df.to_excel('job_summary.xlsx', index=False)
```

Advantages

- Handles very large files
- Lower memory usage
- Scales to production systems

Conclusion

- Use chunking for large files
- Use lazy evaluation for streaming data
- Essential for data engineering & scraping